

Computer Vision

Lecture 04: 객체 감지 모델 Tutorial

Jaesung Lee

curseor@cau.ac.kr

2025. 03. 26.



Contents

1. 환경설정 및 Pascal VOC 데이터셋 다운로드
2. 데이터셋 구조 탐색 및 포맷 변환
3. Train/Test 데이터셋 분할 및 data.yaml 구성
4. YOLOv5 모델 학습
5. 모델 검증 및 추론

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

- 깃허브 리파지토리
 - https://github.com/tkdgur658/CAU_25_Spring_ComputerVision/tree/main/Week04

- 가상 환경 설치(README04.md 참조)

1. 가상환경 생성/활성화

다음 명령어를 통해 Python 3.10을 사용하는 `ComputerVision` 이라는 이름의 가상환경을 생성합니다.

```
conda create -n ComputerVision python=3.10
```



2. 가상환경 활성화

생성한 환경을 활성화합니다.

```
conda activate ComputerVision
```



3. Jupyter Kernel 등록

Jupyter Notebook에서 가상환경을 사용할 수 있도록 커널(Kernel)을 등록합니다.

```
pip install ipykernel  
python -m ipykernel install --user --name ComputerVision --display-name ComputerVision
```



4. 필요 패키지 설치

강의에서 사용하는 필수 패키지를 설치합니다. (`requirements.txt` 파일이 있는 폴더에서 실행하세요.)

```
pip install -r requirements.txt
```



설치가 완료되면 Jupyter Notebook을 실행하여 `ComputerVision` 커널을 선택해 사용하면 됩니다.

1. 환경설정 및 Pascal
VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및
포맷 변환

3. Train/Test 데이터셋
분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

- 실습에 필요한 라이브러리 Import함

1. Library Install& Import

```
[1]: 1 import torch
      2 import torchvision
      3 from torchvision import datasets
      4 import os
      5 import sys
      6 import xml.etree.ElementTree as ET
      7 import glob
      8 import random
      9 import shutil
     10 import numpy as np
     11 import yaml
     12 import cv2
     13 import matplotlib.pyplot as plt
     14 from tqdm.notebook import tqdm
     15 from utils import *
     16
     17 print("Python version :", sys.version)
     18 print("PyTorch version :", torch.__version__)
     19 print("Torchvision version :", torchvision.__version__)
     20 device = "cuda" if torch.cuda.is_available() else "cpu"
     21 print("Using Device :", device)
     22
     23 epochs = 5
     24 batch_size = 4
```

Last executed at 2025-03-26 09:07:25 in 1.30s

Python version : 3.10.16 (main, Dec 11 2024, 16:24:50) [GCC 11.2.0]
PyTorch version : 2.6.0+cu124
Torchvision version : 0.21.0+cu124
Using Device : cuda

1. 환경설정 및 Pascal
VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및
포맷 변환

3. Train/Test 데이터셋
분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

- torchvision.datasets의 VOCDetection을 사용하여 Pascal VOC 데이터셋을 자동으로 다운로드됨
 - 'VOCdevkit' 폴더에 VOC2007가 다운로드됨

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및 포맷 변환

3. Train/Test 데이터셋 분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

2. Pascal VOC 데이터셋 다운로드 (torchvision)

```
•[2]: from torchvision.datasets import VOCDetection

# 로컬(혹은 Colab)에서 사용할 루트 경로 지정
# 예: 현재 디렉토리('.') 아래에 VOCdevkit 폴더가 생성됨
DATA_ROOT = "./"

# VOCDetection으로 train 데이터셋을 다운로드
voc_train = VOCDetection(
    root=DATA_ROOT,
    year="2007",
    image_set="train", # trainval, test 등 가능
    download=True,    # 자동 다운로드
    transform=None,
    target_transform=None
)

print("VOC 2007 trainset 다운로드 완료!")
print("데이터 수:", len(voc_train))
print("저장 경로 구조 확인:", os.listdir(os.path.join(DATA_ROOT, "VOCdevkit")))
```

```
Using downloaded and verified file: ./VOCtrainval_06-Nov-2007.tar
Extracting ./VOCtrainval_06-Nov-2007.tar to ./
VOC 2007 trainset 다운로드 완료!
데이터 수: 2501
저장 경로 구조 확인: ['VOC2007']
```

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

- torchvision.datasets의 VOCDetection을 사용하여 Pascal VOC 데이터셋을 자동으로 다운로드됨
 - 'VOCdevkit' 폴더에 VOC2007가 다운로드됨

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및 포맷 변환

3. Train/Test 데이터셋 분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

📁 / ... / VOCdevkit / VOC2007 /

Name	Modifi...
📁 Annotations	17y ago
📁 ImageSets	17y ago
📁 JPEGImages	17y ago
📁 SegmentationClass	17y ago
📁 SegmentationObject	17y ago

Annotations

- 이미지 내 객체의 정보를 담은 XML 형태의 주석(annotation) 파일들이 들어있는 폴더
- 주로 bounding box 좌표, 클래스 라벨 등 객체의 위치와 종류 정보를 포함

ImageSets

- 데이터셋의 사용 목적(train, val, test 등)에 따라 이미지를 분류한 리스트를 저장한 폴더
- 예를 들어, train.txt, val.txt, test.txt 등의 파일에서 각 이미지의 파일명이 기재되어 있으며, segmentation이나 detection과 같은 용도별 리스트를 별도로 제공

JPEGImages

- 실제 이미지 파일(JPEG 포맷)이 저장된 폴더

SegmentationClass

- 이미지의 Semantic Segmentation을 위한 픽셀 단위 클래스 레이블(mask)이 저장된 폴더
- 색상 인코딩을 이용하여 클래스별로 픽셀의 의미를 구분

SegmentationObject

- Instance-level Segmentation(개별 객체 단위 세그멘테이션)을 위한 픽셀 레이블(mask)이 저장된 폴더
- 개별 객체마다 다른 색상을 부여하여 객체 간 구분을 명확히 하고 PNG 형식 저장

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

- 데이터셋 시각화

1. 환경설정 및 Pascal VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및 포맷 변환

3. Train/Test 데이터셋 분할 및 data.yaml 구성

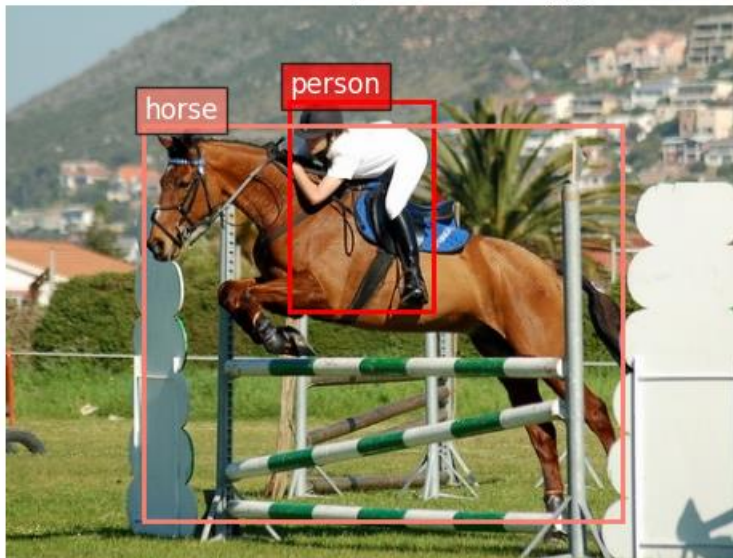
4. YOLOv5 모델 학습

5. 모델 검증 및 추론

```
[3]: 1 fig, ax = visualize_voc_sample(voc_train, 1)
Last executed at 2025-03-26 09:25:37 in 89ms

Annotation structure:
{'annotation': {'folder': 'VOC2007', 'filename': '000017.jpg', 'source': {'database': 'The VOC2007 Database', 'annotation': 'PASCAL VOC2007', 'image': 'flickr', 'flickrid': '228217974'}, 'owner': {'flickrid': 'genewolf', 'name': 'whiskey kitten'}, 'size': {'width': '480', 'height': '364', 'depth': '3'}, 'segmented': '0', 'object': [{'name': 'person', 'pose': 'Left', 'truncated': '0', 'difficult': '0', 'bndbox': {'xmin': '185', 'ymin': '62', 'xmax': '279', 'ymax': '199'}}, {'name': 'horse', 'pose': 'Left', 'truncated': '0', 'difficult': '0', 'bndbox': {'xmin': '90', 'ymin': '78', 'xmax': '403', 'ymax': '336'}}]}}
Image size: (480, 364)
Image filename: 000017.jpg
Image folder: VOC2007
Total objects in this image: 2
```

VOC 2007 Sample 1: 000017.jpg



2. 데이터셋 구조 탐색 및 포맷 변환

- YOLOv5를 학습하려면 이미지와 레이블(.txt)이 필요, 각 이미지마다 같은 이름의 .txt 파일이 필요함 (e.g., 000001.jpg → 000001.txt)
- 이를 위해 다운로드된 데이터셋 내 XML 어노테이션 파일을 txt 형태로 변환

3. Dataset configuration

```
•[4]: 1 # Function to convert VOC XML into YOLO text
      2 # Split train/val/test
      3 # Create a data.yaml file
      4 yolo_dataset = "yolo_voc_dataset"
      5 os.makedirs(yolo_dataset, exist_ok=True)
      6 setup_yolo_dataset(yolo_dataset)
```

Last executed at 2025-03-26 09:49:55 in 625ms

Found 5011 valid images and labels.
Training: 3006 images (60.0%)
Validation: 1002 images (20.0%)
Testing: 1003 images (20.0%)
Created dataset.yaml in yolo_voc_dataset

```
def setup_yolo_dataset(yolo_dataset):
    voc_root = "VOCdevkit/VOC2007"
    voc_annotations = os.path.join(voc_root, "Annotations")
    voc_images = os.path.join(voc_root, "JPEGImages")
    # Create directories
    for split in ["train", "val", "test"]:
        for folder in ["images", "labels"]:
            os.makedirs(os.path.join(yolo_dataset, folder, split), exist_ok=True)

    # YOLO Label save directory
    temp_labels_dir = os.path.join(yolo_dataset, "temp_labels")
    os.makedirs(temp_labels_dir, exist_ok=True)

    # Label conversion for all XML files
    xml_files = glob.glob(os.path.join(voc_annotations, "*.xml"))
    valid_images = []

    for xml_file in xml_files:
        base_name = os.path.splitext(os.path.basename(xml_file))[0]
        img_file = os.path.join(voc_images, f"{base_name}.jpg")

        # Only valid if image file exists and label conversion is successful
        if os.path.exists(img_file) and convert_voc_to_yolo(xml_file, temp_labels_dir):
            valid_images.append(base_name)

    print(f"Found {len(valid_images)} valid images and labels.")
```


3. Train/Test 데이터셋 분할 및 data.yaml 구성

- Pascal VOC는 ImageSets/Main 폴더 내 txt 파일로 이미지 분할(train, val, test)을 관리
- 본 실습에서는 VOC2007 Train 데이터를 Train : Validation : Test= 0.6 : 0.2 : 0.2 비율로 분할하여 사용

```
# train/val/test split (60/20/20)
random.seed(42)
random.shuffle(valid_images)
train_split = int(len(valid_images) * 0.6)
val_split = int(len(valid_images) * 0.8)

train_images = valid_images[:train_split]
val_images = valid_images[train_split:val_split]
test_images = valid_images[val_split:]

print(f"Training: {len(train_images)} images ({len(train_images)/len(valid_images)*100:.1f}%)")
print(f"Validation: {len(val_images)} images ({len(val_images)/len(valid_images)*100:.1f}%)")
print(f"Testing: {len(test_images)} images ({len(test_images)/len(valid_images)*100:.1f}%)")

# Copy image and label files
for img_set, subset in [(train_images, "train"), (val_images, "val"), (test_images, "test")]:
    for img_name in img_set:
        # Copy image
        src_img = os.path.join(voc_images, f"{img_name}.jpg")
        dst_img = os.path.join(yolo_dataset, "images", subset, f"{img_name}.jpg")
        shutil.copy(src_img, dst_img)

# Copy label
src_label = os.path.join(temp_labels_dir, f"{img_name}.txt")
dst_label = os.path.join(yolo_dataset, "labels", subset, f"{img_name}.txt")
if os.path.exists(src_label):
    shutil.copy(src_label, dst_label)
```

3. Train/Test 데이터셋 분할 및 data.yaml 구성

- YOLOv5가 참고할 data.yaml을 작성해야 함
- Ultralytics에서 권장하는 방식으로, 'train' 키에 train.txt 경로, 'val' 키에 val.txt 경로를 기입하여 진행
- 클래스 이름은 Pascal VOC 20개로 설정

```
# Create dataset.yaml file for YOLOv5 training
yaml_content = {
    'path': os.path.abspath(yolo_dataset),
    'train': 'images/train',
    'val': 'images/val',
    'test': 'images/test',
    'nc': len(voc_classes),
    'names': voc_classes
}

with open(os.path.join(yolo_dataset, 'dataset.yaml'), 'w') as f:
    yaml.dump(yaml_content, f, default_flow_style=False)

print(f"Created dataset.yaml in {yolo_dataset}")
```

```
# 클래스 명
names:
  0: aeroplane
  1: bicycle
  2: bird
  3: boat
  4: bottle
  5: bus
  6: car
  7: cat
  8: chair
  9: cow
  10: diningtable
  11: dog
  12: horse
  13: motorbike
  14: person
  15: pottedplant
  16: sheep
  17: sofa
  18: train
  19: tvmonitor
"""
```

4. YOLOv5 모델 학습

- YOLOv5 모델 구조와 사전학습 가중치를 로드
- Yaml 파일 경로, Epochs 수, batch 사이즈, 입력 이미지 크기 등을 설정하고 학습 진행
- 사전에 분류된 항목별로 분류된 결과와 성능측정 결과를 확인

4. Training YOLOv5 (Ultralytics)

```
[5]: 1 from ultralytics import YOLO
      2 # Load model
      3 model = YOLO("yolov5s.pt")
      4 print("YOLOv5 model loading complete!")
      5 # Training - using GPU
      6 model.train(
      7     data=os.path.join(yolo_dataset, "dataset.yaml"),
      8     epochs=epochs,
      9     batch=batch_size,
     10     imgsz=416,
     11     name="yolov5_voc_demo",
     12     device=device,
     13     workers=max(1, os.cpu_count() - 1)
     14 )
```

4. YOLOv5 모델 학습

- YOLOv5 모델 구조와 사전학습 가중치를 로드
- Yaml 파일 경로, Epochs 수, batch 사이즈, 입력 이미지 크기 등을 설정하고 학습 진행
- 사전에 분류된 항목별로 분류된 결과와 성능측정 결과를 확인

📁 / ... / detect / yolov5_voc_demo /

Name	Modifi...
📁 weights	2m ago
📄 args.yaml	3m ago
📄 confusion_matrix_normalized.png	1m ago
📄 confusion_matrix.png	1m ago
📄 F1_curve.png	1m ago
📄 labels_correlogram.jpg	3m ago
📄 labels.jpg	3m ago
📄 P_curve.png	1m ago
📄 PR_curve.png	1m ago
📄 R_curve.png	1m ago
📄 results.csv	1m ago
📄 results.png	1m ago
📄 train_batch0.jpg	3m ago
📄 train_batch1.jpg	3m ago
📄 train_batch2.jpg	3m ago
📄 val_batch0_labels.jpg	1m ago
📄 val_batch0_pred.jpg	1m ago
📄 val_batch1_labels.jpg	1m ago
📄 val_batch1_pred.jpg	1m ago
📄 val_batch2_labels.jpg	1m ago
📄 val_batch2_pred.jpg	1m ago

5. 모델 검증 및 추론

- 학습 결과로 나온 .pt 파일을 불러와서 검증 과정 진행
- 이미지 디렉토리에서 이미지 파일 목록을 가져오고 첫 번째 이미지 파일을 사용하여 모델로 예측 수행
- 성능 지표 확인

5. Inference

```
1 from ultralytics import YOLO
2 # Load the best trained model
3 trained_model = YOLO("runs/detect/yolov5_voc_demo/weights/best.pt")
4
5 conf = 0.5
6 # Run inference on test dataset
7 print("Running inference on test dataset...")
8 test_results = trained_model.val(
9     data=os.path.join(yolo_dataset, "dataset.yaml"),
10     split="test", # Specify test split
11     imgsz=416,
12     batch=batch_size,
13     device=device,
14     conf=conf, # 신뢰도 임계값 설정 (0.0 ~ 1.0)
15     save_json=True, # Save results in COCO JSON format
16     save_conf=True, # Save confidence scores
17     plots=True, # Generate performance plots
18     verbose=False
19 )
20
21 print("\nPerformance Measures on Test Dataset:")
22 measures = test_results.box
23
24 print(f"mAP@0.5 : {measures.map50:.6f}")
25 print(f"mAP@0.5:0.95 : {measures.map:.6f}")
26
27 if isinstance(measures.p, np.ndarray):
28     print(f"Precision : {np.mean(measures.p):.6f} (mean)")
29     print(f"Recall : {np.mean(measures.r):.6f} (mean)")
30     print(f"F1-Score : {np.mean(measures.f1):.6f} (mean)")
31 else:
32     print(f"Precision : {measures.p:.6f}")
33     print(f"Recall : {measures.r:.6f}")
34     print(f"F1-Score : {measures.f1:.6f}")
35
36 print(f"\nDetailed results saved to: {test_results.save_dir}")
37
38 print("\nRunning inference on individual test images...")
39 test_image_dir = os.path.join(yolo_dataset, "images/test")
40 output_dir = "test_predictions"
41 run_individual_inference(trained_model, test_image_dir, output_dir, conf=conf)
42
43 print(f"\nInference results saved to: {output_dir}")
```

Running inference on test dataset...

Ultralytics 8.3.96 Python-3.10.16 torch-2.6.0+cu124 CUDA:0 (NVIDIA GeForce RTX 3090, 24253MiB)
YOLOv5s summary (fused): 84 layers, 9,119,276 parameters, 0 gradients, 23.9 GFLOPs

val: Scanning /home/sanghyuck/Desktop/LSH_74/CAU_25_Spring_ComputerVision/Week04

Class	Images	Instances	Box(P)	R	mAP50	m
all	1003	2537	0.749	0.685	0.75	0.566

Speed: 0.1ms preprocess, 0.9ms inference, 0.0ms loss, 0.2ms postprocess per image

Saving runs/detect/val5/predictions.json...

Results saved to runs/detect/val5

Performance Measures on Test Dataset:

mAP@0.5 : 0.750055
mAP@0.5:0.95 : 0.565563
Precision : 0.749217 (mean)
Recall : 0.685183 (mean)
F1-Score : 0.713744 (mean)

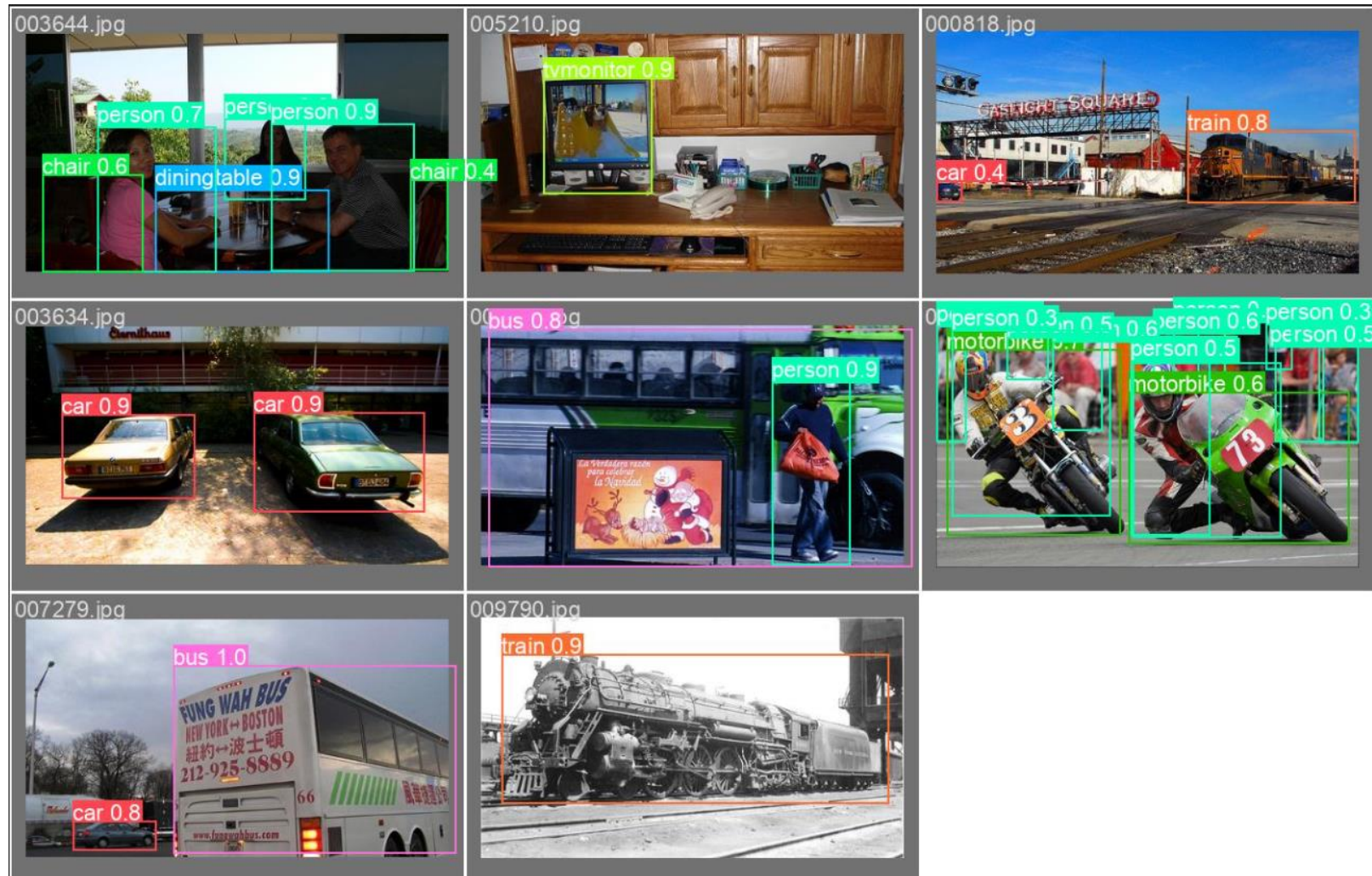
Detailed results saved to: runs/detect/val5

Running inference on individual test images...

Inference results saved to: test_predictions

5. 모델 검증 및 추론

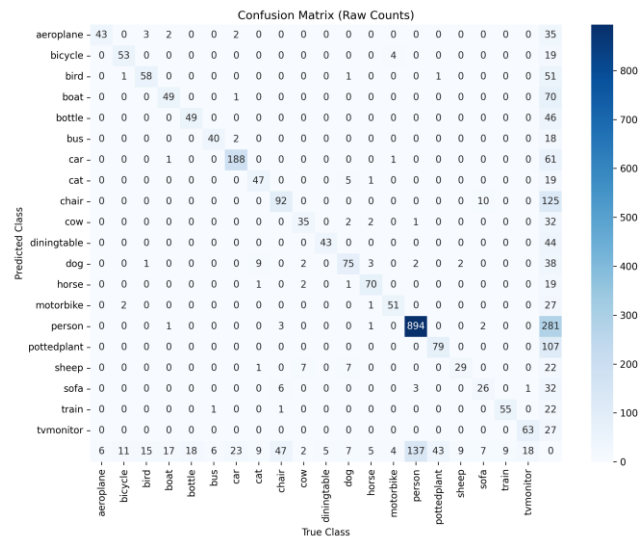
- 시각화를 통해 이미지 데이터에서 처리된 객체 감지 결과를 확인



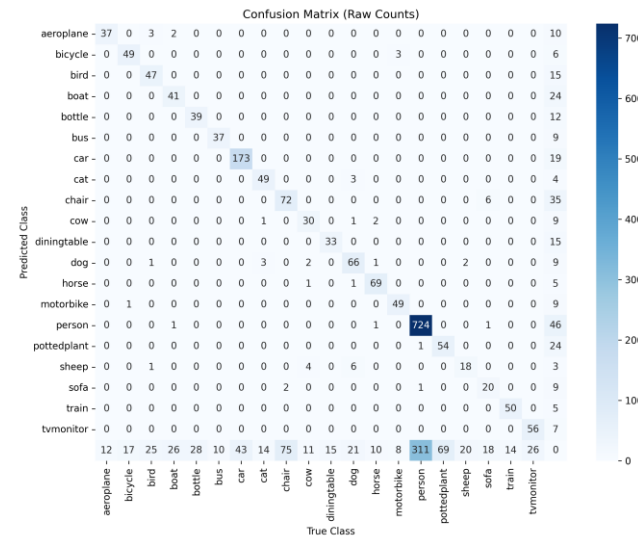
5. 모델 검증 및 추론

- 구글드라이브(<https://drive.google.com/drive/folders/1KCxBWxurjTrliPfGEcQH8QgbLzPzlWsb?usp=sharing>)
 - test_predictions_0.2.zip, test_predictions_0.5.zip, test_predictions_0.8.zip
- Confusion Matrix를 통해 에러 분석
 - 서로 다른 Confidence Threshold 적용
 - 0.2인 경우는 많은 Target을 발견 But 배경을 객체로 감지(낮은 Precision, 높은 Recall)
 - 0.8인 경우는 False Positive 감소 But 객체 인식률 감소(높은 Precision, 낮은 Recall)

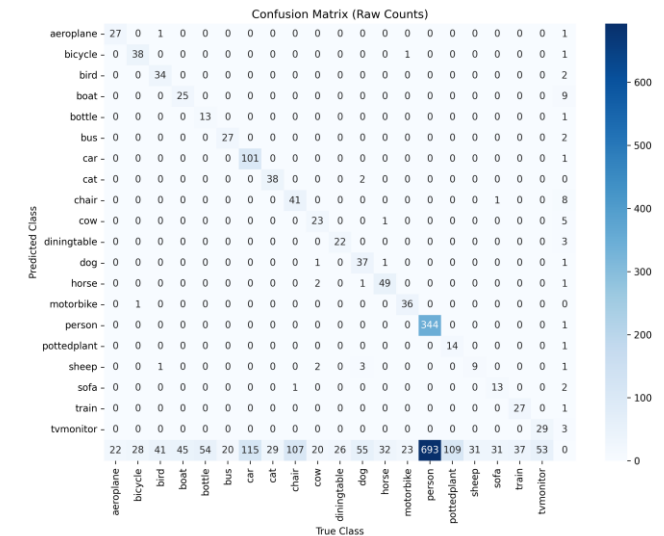
IoU=0.2



IoU=0.5



IoU=0.8



5. 모델 검증 및 추론

- 구글드라이브(<https://drive.google.com/drive/folders/1KCxBWxurjTrliPfGEcQH8QgbLzPzlWsb?usp=sharing>)
 - prediction_analysis.zip
- 모델 예측 결과 : 실선
 - 초록색 사각형 (Green): True Positive (TP) 예측
 - 모델이 올바르게 감지한 객체
 - 빨간색 사각형 (Red): False Positive (FP) 예측
 - 모델이 잘못 감지한 객체 또는 정확도가 낮은 예측
- 실제 정답 (Ground Truth) : 점선
 - 파란색 점선 사각형 (Blue): 매칭된 Ground Truth
 - 모델이 성공적으로 감지한 실제 정답 객체
 - 노란색 점선 사각형 (Yellow): False Negative (FN) Ground Truth
 - 모델이 감지하지 못한 실제 정답 객체

1. 환경설정 및 Pascal
VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및
포맷 변환

3. Train/Test 데이터셋
분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

5. 모델 검증 및 추론

- 구글드라이브(<https://drive.google.com/drive/folders/1KCxBWxurjTrliPfGEcQH8QgbLzPzlWsb?usp=sharing>)
 - prediction_analysis.zip
- 모델 예측 결과 : 실선
 - 초록색 사각형 (Green): True Positive (TP) 예측
 - 모델이 올바르게 감지한 객체
 - 빨간색 사각형 (Red): False Positive (FP) 예측
 - 모델이 잘못 감지한 객체 또는 정확도가 낮은 예측
- 실제 정답 (Ground Truth) : 점선
 - 파란색 점선 사각형 (Blue): 매칭된 Ground Truth
 - 모델이 성공적으로 감지한 실제 정답 객체
 - 노란색 점선 사각형 (Yellow): False Negative (FN) Ground Truth
 - 모델이 감지하지 못한 실제 정답 객체

1. 환경설정 및 Pascal
VOC 데이터셋 다운로드

2. 데이터셋 구조 탐색 및
포맷 변환

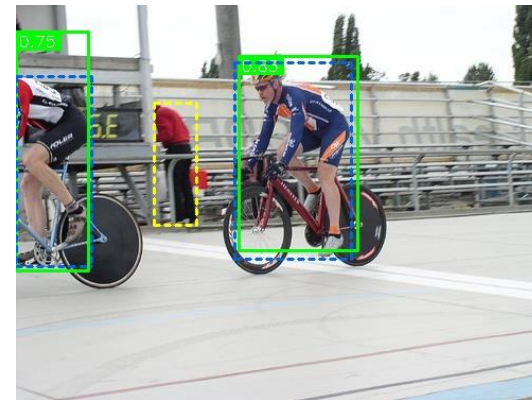
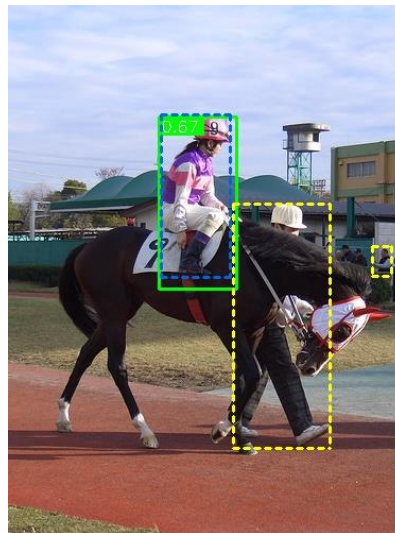
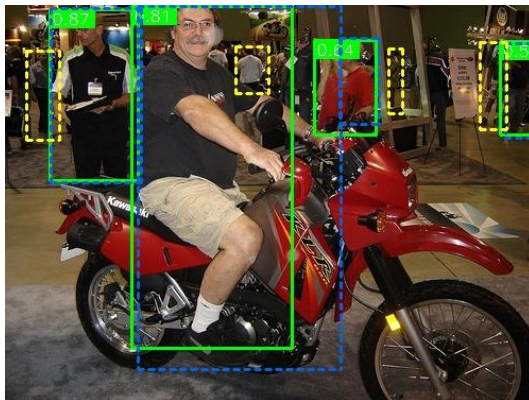
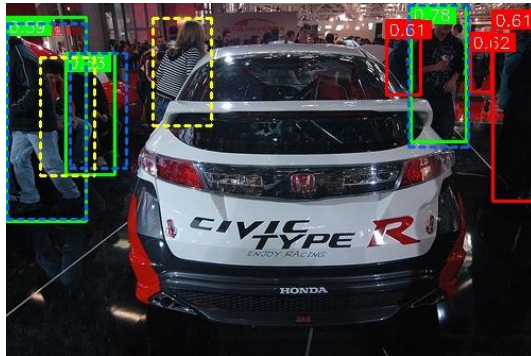
3. Train/Test 데이터셋
분할 및 data.yaml 구성

4. YOLOv5 모델 학습

5. 모델 검증 및 추론

5. 모델 검증 및 추론

- 노란색 점선과 파란색 점선과의 비교로 가설 설정



Thank you